

Taller Práctico de Programación Orientada a Objetos en Java

Sistema de Gestión de un Concesionario de Vehículos

Docente: Ing. William Alexander Matallana Porras

1. Contexto del Ejercicio

La concesionaria "AutoMotion S.A.S." vende vehículos de distintos tipos: automóviles, motocicletas y camiones de carga. Actualmente el inventario se controla en hojas de cálculo, lo que genera errores en el cálculo del precio final de cada vehículo (que varía según el tipo, por impuestos y recargos específicos) y dificulta consultar el valor total del inventario disponible.

La gerencia les ha contratado como desarrolladores para construir la primera versión de un sistema en Java que organice el inventario, calcule automáticamente el precio final de cada vehículo según su tipo y permita consultar el valor total de las unidades disponibles para la venta.

Este ejercicio busca que apliquen, en un caso realista, los cuatro principios de la Programación Orientada a Objetos trabajados en la sesión anterior: encapsulamiento, abstracción, herencia y polimorfismo.

2. Objetivos de Aprendizaje

- Aplicar encapsulamiento para proteger el estado interno de los objetos mediante atributos privados y métodos de acceso controlado.
- Diseñar una clase abstracta que represente el concepto general de un vehículo, ocultando los detalles específicos de cada tipo.
- Implementar herencia para reutilizar atributos y comportamientos comunes entre distintos tipos de vehículos.
- Aplicar polimorfismo para que un mismo método se comporte de forma distinta según el tipo real del objeto.
- Gestionar desde la clase Main una colección mixta de vehículos y las operaciones básicas del inventario.
- Construir un programa Java funcional, compilable y ejecutable que integre los cuatro principios en un solo sistema.

3. Descripción General del Sistema

El sistema debe representar tres tipos de vehículos que comparten características comunes (placa, marca, modelo, año y precio base), pero que se diferencian en atributos específicos y, sobre todo, en la forma de calcular su precio final. Todos los tipos de vehículo deben derivar de una misma clase abstracta.

La clase Main será responsable de crear y administrar una colección mixta de vehículos, listar el inventario, calcular su valor total y buscar unidades por placa, sin requerir una clase gestora adicional.

4. Diseño de Clases Requerido

4.1 Clase abstracta Vehiculo

Es la clase base de la jerarquía. No debe poder instanciarse directamente.

Atributo	Tipo	Visibilidad	Restricción / validación
placa	String	private	No puede estar vacía
marca	String	private	No puede estar vacía
modelo	String	private	No puede estar vacío
anio	int	private	Entre 1990 y el año actual
precioBase	double	private	Debe ser mayor a 0

Métodos requeridos:

- Constructor que reciba e inicialice los cinco atributos, validando las restricciones anteriores.
- Getters y setters para cada atributo, aplicando las validaciones en los setters (si el valor no es válido, el atributo no se modifica y se informa por consola).
- Método abstracto: `public abstract double calcularPrecioFinal();`
- Método concreto: `public String mostrarFicha();` que retorne un texto con los datos comunes del vehículo (puede ser reutilizado por las subclasses).

4.2 Clase Automovil (extiende Vehiculo)

Atributo adicional	Tipo	Descripción
numeroPuertas	int	Entre 2 y 5
tipoCombustible	String	"Gasolina", "Diésel" o "Eléctrico"

Regla de negocio: `calcularPrecioFinal()` debe retornar el `precioBase` más un 8% de impuesto de matrícula. Si el `tipoCombustible` es "Eléctrico", el impuesto se reduce al 4% como incentivo ambiental.

4.3 Clase Motocicleta (extiende Vehiculo)

Atributo adicional	Tipo	Descripción
cilindraje	int	Cilindraje en centímetros cúbicos (c.c.)

Regla de negocio: `calcularPrecioFinal()` debe retornar el `precioBase` más un 3% de impuesto de matrícula. Si el cilindraje supera 500 c.c., se suma un recargo adicional del 2% por alto cilindraje.

4.4 Clase CamionCarga (extiende Vehiculo)

Atributo adicional	Tipo	Descripción
capacidadToneladas	double	Capacidad máxima de carga

Regla de negocio: calcularPrecioFinal() debe retornar el precioBase más un 5% de impuesto, más un recargo fijo de \$500.000 COP por cada tonelada de capacidad.

4.5 Clase Main (gestión del inventario)

Toda la gestión del inventario se realizará directamente en Main.java. La clase Main debe declarar una colección de vehículos, por ejemplo:

```
ArrayList<Vehiculo> inventario = new ArrayList<>();
```

Dentro de Main se deben implementar las siguientes operaciones como métodos static:

- agregarVehiculo(ArrayList<Vehiculo> inventario, Vehiculo v): añade un vehículo a la colección.
- listarInventario(ArrayList<Vehiculo> inventario): imprime la ficha y el precio final de cada vehículo, recorriendo la lista y usando polimorfismo, sin preguntar el tipo de cada objeto.
- calcularValorTotalInventario(ArrayList<Vehiculo> inventario): retorna la suma del precio final de todos los vehículos.
- buscarPorPlaca(ArrayList<Vehiculo> inventario, String placa): retorna el vehículo con esa placa, o null si no existe.

El método main() debe crear el ArrayList, construir los objetos, invocar estos métodos y mostrar los resultados. No se permite crear una clase adicional para administrar el inventario.

5. Requisitos por Principio de POO

Su implementación será revisada verificando específicamente cada uno de estos cuatro principios:

Principio	Debe evidenciarse en...
Encapsulamiento	Atributos private en Vehiculo y en sus subclases, con getters/setters que validen los datos antes de asignarlos.
Abstracción	La clase Vehiculo es abstracta y declara al menos un método abstracto que las subclases están obligadas a implementar.
Herencia	Automovil, Motocicleta y CamionCarga extienden Vehiculo y reutilizan sus atributos y métodos comunes mediante super().
Polimorfismo	Main.listarInventario() recorre una lista de tipo Vehiculo y cada objeto ejecuta su propia versión de calcularPrecioFinal().

6. Funcionalidades que Debe Tener el Programa (clase Main)

La clase Main.java debe ejecutar, en orden, la siguiente demostración:

- 1 Crear un ArrayList<Vehiculo> que represente el inventario.
- 2 Crear al menos 2 objetos de cada tipo de vehículo (2 automóviles, 2 motocicletas y 2 camiones), usando datos distintos y realistas.
- 3 Agregar todos los vehículos al ArrayList mediante el método estático agregarVehiculo() definido en Main.
- 4 Invocar listarInventario() y mostrar la ficha y el precio final de cada vehículo.
- 5 Mostrar el valor total del inventario usando calcularValorTotalInventario().
- 6 Buscar un vehículo por placa con buscarPorPlaca() y mostrar su información.
- 7 Intentar crear un vehículo con un dato inválido (por ejemplo, año 1800 o precioBase negativo) y mostrar por consola el mensaje de validación, demostrando que el encapsulamiento protege el objeto.

7. Ejemplo de Salida Esperada (formato de referencia)

El contenido exacto puede variar según sus datos de prueba; lo importante es que la estructura y el comportamiento coincidan con el siguiente formato:

=== INVENTARIO AUTOMOTION S.A.S. ===

Placa: ABC123 | Marca: Mazda | Modelo: 3 | Año: 2022
Tipo: Automóvil | Puertas: 4 | Combustible: Gasolina
Precio final: \$37.800.000

Placa: XYZ789 | Marca: Yamaha | Modelo: MT-07 | Año: 2023
Tipo: Motocicleta | Cilindraje: 689 c.c.
Precio final: \$28.350.000

Placa: CAM456 | Marca: Kenworth | Modelo: T800 | Año: 2021
Tipo: Camión de carga | Capacidad: 12.0 ton
Precio final: \$184.500.000

=== VALOR TOTAL DEL INVENTARIO: [valor según los seis vehículos] ===

--- Prueba de validación ---

Error: el año debe estar entre 1990 y el año actual. No se creó el vehículo.

8. Entregables

- Código fuente completo en Java, organizado en archivos separados por clase: Vehiculo.java, Automovil.java, Motocicleta.java, CamionCarga.java y Main.java. No debe incluirse una clase Concesionario.
- El proyecto debe compilar y ejecutarse sin errores.
- Comentarios tipo Javadoc en cada clase indicando qué principio(s) de POO aplica y por qué.
- Un diagrama de clases simple (a mano, en herramienta UML o incluso en texto) que muestre la jerarquía de herencia Vehiculo → Automovil / Motocicleta / CamionCarga. Puede agregarse Main como clase que utiliza una colección de Vehiculo.
- Entrega en un archivo .zip o repositorio Git con el nombre: ApellidoNombre_TallerPOO.

9. Recomendaciones Finales

- Empiecen diseñando la clase Vehiculo antes que las subclases: si la base está bien definida, el resto es más sencillo.
- Prueben cada clase por separado antes de integrarla con la colección administrada desde Main.
- Definan en Main métodos estáticos pequeños y con una sola responsabilidad para evitar concentrar toda la lógica dentro del método main().
- Usen nombres de variables y métodos en español o inglés, pero sean consistentes en todo el proyecto.
- No dejen atributos públicos “por comodidad”: es exactamente el error de encapsulamiento que vimos en la presentación anterior.
- No utilicen instanceof para listar o calcular precios: el objetivo es que cada objeto ejecute polimórficamente su propia versión de calcularPrecioFinal().